

**LAPORAN PRAKTIKUM**  
**ALGORITMA PEMROGRAMAN 2**  
**MODUL 5 – REKURSIF**



**NAMA : MOH. CHANDRA WARDANA**

**NIM : 109082530025**

**KELAS S1IF-13-06**

**PROGRAM STUDI TEKNIK INFORMATIKA**

**FAKULTAS INFORMATIKA**

**TELKOM UNIVERSITY PURWOKERTO**

**2026**

## A. Dasar Teori

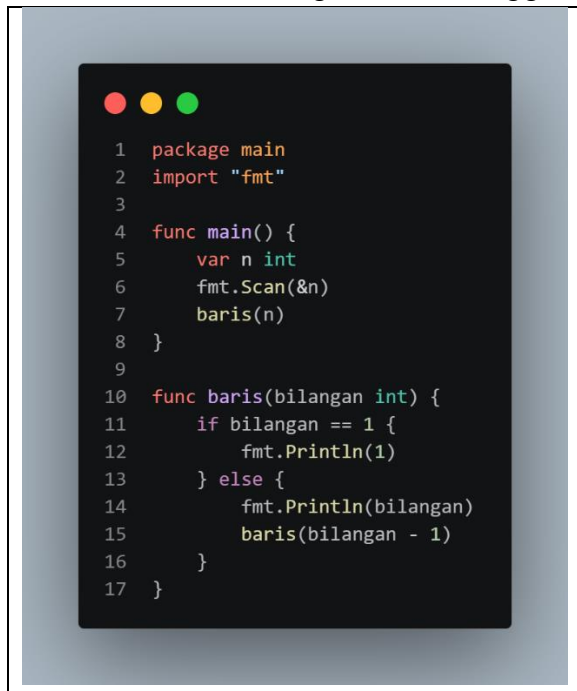
Rekursif adalah teknik dalam pemrograman di mana sebuah fungsi memanggil dirinya sendiri untuk menyelesaikan suatu masalah. Konsep utama dari rekursif adalah memecah masalah besar menjadi sub-masalah yang lebih kecil dan serupa hingga mencapai kondisi tertentu yang disebut base-case. Tanpa adanya base-case, fungsi rekursif akan berjalan tanpa henti (infinite loop) sehingga menyebabkan error seperti stack overflow.

Dalam rekursif terdapat dua komponen penting yaitu base-case dan recursive-case. Base-case adalah kondisi berhentinya proses rekursif, sedangkan recursive-case adalah bagian di mana fungsi memanggil dirinya sendiri dengan parameter yang lebih kecil atau mendekati kondisi berhenti. Proses rekursif juga memiliki dua fase yaitu forward (pemanggilan fungsi terus menerus) dan backward (kembali setelah base-case tercapai).

Rekursif sering digunakan untuk menyelesaikan masalah seperti faktorial, fibonacci, pencarian nilai, dan traversal struktur data. Walaupun semua rekursif dapat diubah menjadi iteratif (loop), rekursif sering membuat kode lebih sederhana dan mudah dipahami. Namun, penggunaan rekursif harus hati-hati karena bisa memakan memori lebih besar dibandingkan iterasi.

## B. Guided

### 1. Baris Bilangan dari n hingga 1

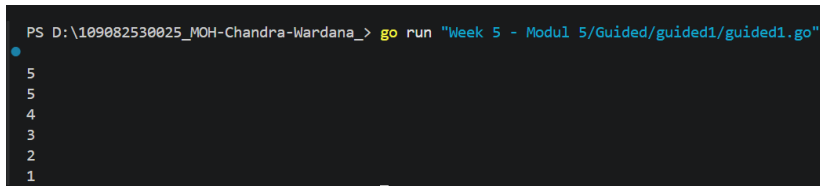


```
1 package main
2 import "fmt"
3
4 func main() {
5     var n int
6     fmt.Scan(&n)
7     baris(n)
8 }
9
10 func baris(bilangan int) {
11     if bilangan == 1 {
12         fmt.Println(1)
13     } else {
14         fmt.Println(bilangan)
15         baris(bilangan - 1)
16     }
17 }
```

Penjelasan code :

Program ini dibuat untuk menampilkan deret bilangan dari nilai n hingga 1 menggunakan konsep rekursif, di mana pada fungsi main program menerima input dari user lalu memanggil fungsi baris dengan parameter n tersebut. Di dalam fungsi baris terdapat kondisi base-case yaitu ketika nilai bilangan sama dengan 1, maka program hanya mencetak angka 1 dan tidak melakukan pemanggilan fungsi kembali sehingga proses rekursif berhenti. Sedangkan pada kondisi recursive-case, yaitu ketika bilangan tidak sama dengan 1, program akan mencetak nilai bilangan saat ini terlebih dahulu lalu memanggil kembali fungsi baris dengan parameter bilangan-1, sehingga setiap pemanggilan fungsi akan mengurangi nilai bilangan secara bertahap sampai mencapai 1. Struktur ini menunjukkan bahwa proses cetak dilakukan pada saat forward sebelum pemanggilan berikutnya.

Output :



```
PS D:\109082530025_MOH-Chandra-Wardana> go run "Week 5 - Modul 5/Guided/guided1/guided1.go"
5
4
3
2
1
```

Penjelasan output :

Jika user memasukkan nilai 5, maka program akan menjalankan fungsi baris(5) yang pertama-tama mencetak angka 5 lalu memanggil baris(4), kemudian mencetak 4 dan memanggil baris(3), dan seterusnya hingga mencapai baris(1) yang merupakan base-case. Karena pada base-case hanya mencetak 1 tanpa memanggil fungsi lagi, maka proses berhenti di situ. Akibatnya output yang dihasilkan adalah 5 4 3 2 1 secara berurutan menurun, karena setiap nilai dicetak sebelum pemanggilan rekursif berikutnya dilakukan.

## 2. Penjumlahan 1 hingga n

```

1 package main
2
3 import "fmt"
4
5 func main() {
6     var n int
7     fmt.Scan(&n)
8     fmt.Println(penjumlahan(n))
9 }
10
11 func penjumlahan(n int) int {
12     if n == 1 {
13         return 1
14     } else {
15         return n + penjumlahan(n-1)
16     }
17 }

```

Penjelasan code :

Program ini digunakan untuk menghitung jumlah bilangan dari 1 sampai n menggunakan fungsi rekursif bernama penjumlahan, di mana nilai n diinput oleh user pada fungsi main kemudian hasil dari fungsi tersebut langsung ditampilkan. Dalam fungsi penjumlahan terdapat base-case yaitu ketika n bernilai 1, maka fungsi akan mengembalikan nilai 1 sebagai dasar perhitungan. Sedangkan pada recursive-case, fungsi akan mengembalikan hasil dari n ditambah dengan pemanggilan fungsi penjumlahan(n-1), sehingga setiap pemanggilan akan mengurangi nilai n secara bertahap hingga mencapai 1. Proses ini terjadi secara forward hingga base-case, lalu dilanjutkan backward untuk menjumlahkan semua nilai.

Output :

```

PS D:\109082530025_MOH-Chandra-Wardana> go run "Week 5 - Modul 5/Guided/guided2/guided2.go"
5
15

```

Penjelasan output :

Jika input yang diberikan adalah 5, maka proses yang terjadi adalah penjumlahan(5) akan memanggil penjumlahan(4), kemudian penjumlahan(3), hingga penjumlahan(1) yang mengembalikan nilai 1. Setelah itu, proses backward terjadi dengan menjumlahkan hasilnya secara berurutan yaitu 2+1, lalu 3+2, lalu 4+3, hingga akhirnya 5+10 sehingga totalnya adalah 15. Oleh karena itu, output yang ditampilkan program adalah 15 yang merupakan hasil dari penjumlahan semua bilangan dari 1 sampai 5.

### 3. Pangkat $2^n$



```
1 package main
2 import "fmt"
3
4 func main() {
5     var n int
6     fmt.Scan(&n)
7     fmt.Println(pangkat(n))
8 }
9
10 func pangkat(n int) int {
11     if n == 0 {
12         return 1
13     } else {
14         return 2 * pangkat(n-1)
15     }
16 }
```

Penjelasan code

:

Program ini digunakan untuk menghitung nilai perpangkatan 2 pangkat n menggunakan rekursif, di mana nilai n diinput oleh user kemudian diproses oleh fungsi pangkat. Di dalam fungsi tersebut terdapat base-case yaitu ketika n bernilai 0, maka fungsi akan mengembalikan nilai 1 karena secara matematis setiap bilangan berpangkat nol bernilai 1. Sedangkan pada recursive-case, fungsi akan mengembalikan hasil dari 2 dikalikan dengan pemanggilan fungsi pangkat(n-1), sehingga setiap pemanggilan akan mengurangi nilai n hingga mencapai 0. Struktur ini menunjukkan bahwa proses perkalian akan terjadi pada fase backward setelah semua pemanggilan selesai.

Output :

```
PS D:\109082530025_MOH-Chandra-Wardana_> go run "Week 5 - Modul 5/Guided/guided3/guided3.go"
3
8
```

Penjelasan output :

Jika user memasukkan nilai 3, maka program akan menghitung pangkat(3) yang memanggil pangkat(2), lalu pangkat(1), hingga pangkat(0) yang mengembalikan nilai 1. Setelah itu, hasil dikembalikan secara bertahap yaitu  $2 \times 1$ , lalu  $2 \times 2$ , lalu  $2 \times 4$  hingga menghasilkan nilai akhir 8. Dengan demikian output yang ditampilkan adalah 8, yang merupakan hasil dari 2 pangkat 3.

### C. Unguided

#### 1. Soal Unguided 1

Deret Fibonacci

Jawaban :

```
1 package main
2 import "fmt"
3
4 func main() {
5     var n int
6     fmt.Scan(&n)
7     for i := 0; i <= n; i++ {
8         fmt.Print(fibo(i), " ")
9     }
10 }
11
12 func fibo(n int) int {
13     if n == 0 {
14         return 0
15     } else if n == 1 {
16         return 1
17     } else {
18         return fibo(n-1) + fibo(n-2)
19     }
20 }
```

Penjelasan code :

Program ini digunakan untuk menampilkan deret Fibonacci hingga suku ke-n dengan menggunakan fungsi rekursif fibo, di mana fungsi main melakukan

perulangan dari 0 hingga n untuk memanggil fungsi fibo dan mencetak setiap hasilnya. Dalam fungsi fibo terdapat dua base-case yaitu ketika n sama dengan 0 mengembalikan 0 dan ketika n sama dengan 1 mengembalikan 1, karena kedua nilai tersebut adalah dasar dari deret Fibonacci. Untuk nilai lainnya digunakan recursive-case yaitu menjumlahkan hasil dari fibo(n-1) dan fibo(n-2), sehingga setiap nilai bergantung pada dua nilai sebelumnya dan proses ini berlangsung hingga mencapai base-case.

Output :

```
PS D:\109082530025_MOH-Chandra-Wardana_\Week 5 - Modul 5\Unguided\unguided1> go run unguided
1.go
5
0 1 1 2 3 5
PS D:\109082530025_MOH-Chandra-Wardana_\Week 5 - Modul 5\Unguided\unguided1> 
```

Penjelasan :

Jika input yang diberikan adalah 5, maka program akan menghitung nilai fibo untuk setiap indeks mulai dari 0 sampai 5 yaitu fibo(0)=0, fibo(1)=1, fibo(2)=1, fibo(3)=2, fibo(4)=3, dan fibo(5)=5. Hasil tersebut kemudian dicetak secara berurutan sehingga output yang dihasilkan adalah 0 1 1 2 3 5, di mana setiap angka merupakan hasil penjumlahan dari dua angka sebelumnya sesuai dengan aturan deret Fibonacci.

## 2. Soal Unguided 2

Faktor Bilangan

Jawaban :

```
1 package main
2 import "fmt"
3
4 func main() {
5     var n int
6     fmt.Scan(&n)
7     faktor(n, 1)
8 }
9
10 func faktor(n int, i int) {
11     if i > n {
12         return
13     }
14     if n%i == 0 {
15         fmt.Print(i, " ")
16     }
17     faktor(n, i+1)
18 }
```

Penjelasan code :

Program ini digunakan untuk mencari dan menampilkan semua faktor dari suatu bilangan n dengan menggunakan rekursif, di mana fungsi faktor memiliki dua parameter yaitu n sebagai bilangan utama dan i sebagai nilai pembagi yang dimulai dari 1. Base-case dari fungsi ini adalah ketika nilai i lebih besar dari n, maka fungsi akan berhenti tanpa melakukan apa-apa. Sedangkan pada recursive-case, fungsi akan mengecek apakah n habis dibagi i menggunakan operasi modulus, jika iya maka nilai i dicetak sebagai faktor, kemudian fungsi memanggil dirinya kembali dengan nilai i+1 untuk memeriksa pembagi berikutnya hingga mencapai n.

Output :

```
PS D:\109082530025_MOH-Chandra-Wardana> cd "Week 5 - Modul 5\Unguided\unguided2"
PS D:\109082530025_MOH-Chandra-Wardana\Week 5 - Modul 5\Unguided\unguided2> go run u
nguided2.go
12
1 2 3 4 6 12
PS D:\109082530025_MOH-Chandra-Wardana\Week 5 - Modul 5\Unguided\unguided2>
```

Penjelasan :

Jika input yang diberikan adalah 12, maka program akan mengecek semua bilangan dari 1 sampai 12 apakah dapat membagi 12 tanpa sisa, yaitu 1, 2, 3, 4, 6, dan 12. Setiap nilai yang memenuhi kondisi tersebut akan dicetak, sehingga output yang dihasilkan adalah 1 2 3 4 6 12 yang merupakan semua faktor dari bilangan 12.

### 3. Soal Unguided 3

Bilangan Ganjil

Jawaban :

```
[
  package main
  import "fmt"
  func main() {
    var n int
    fmt.Scan(&n)
    ganjil(n)
  }
  func ganjil(n int) {
    if n == 1 {
      fmt.Print(1)
    } else {
      ganjil(n-1)
      if n%2 != 0 {
        fmt.Print(" ", n)
      }
    }
  }
]
```

Penjelasan code :

Program ini bertujuan untuk menampilkan bilangan ganjil dari 1 hingga n menggunakan rekursif, di mana fungsi ganjil akan terus memanggil dirinya sendiri dengan parameter n-1 hingga mencapai base-case yaitu n sama dengan 1 yang langsung mencetak angka 1. Setelah mencapai base-case, proses akan masuk ke fase backward di mana setiap nilai n diperiksa apakah merupakan bilangan ganjil menggunakan kondisi  $n \% 2 \neq 0$ , dan jika benar maka nilai tersebut akan dicetak. Dengan demikian, pencetakan dilakukan saat proses kembali dari rekursi.

Output :

```
PS D:\109082530025_MOH-Chandra-Wardana_\Week 5 - Modul 5\Unguided\unguided3> go run u
nguided3.go
10
1 3 5 7 9
```

Penjelasan :

Jika input yang diberikan adalah 10, maka fungsi akan berjalan dari 10 hingga 1 melalui pemanggilan rekursif, kemudian saat kembali ke atas akan mencetak angka-angka yang memenuhi kondisi ganjil yaitu 1, 3, 5, 7, dan 9. Oleh karena itu output yang dihasilkan adalah 1 3 5 7 9 yang merupakan semua bilangan ganjil dari 1 sampai 10.

#### D. Kesimpulan

Rekursif adalah metode pemrograman yang sangat powerful untuk menyelesaikan masalah yang dapat dipecah menjadi bagian-bagian kecil yang serupa. Dengan memahami konsep **base-case** dan **recursive-case**, kita dapat membuat berbagai program seperti penjumlahan, pangkat, fibonacci, hingga pencarian faktor dengan lebih sederhana.

Namun, penggunaan rekursif harus bijak karena dapat menyebabkan penggunaan memori yang besar jika tidak dirancang dengan baik. Oleh karena itu, penting untuk memastikan bahwa setiap fungsi rekursif memiliki kondisi berhenti yang jelas agar program tidak berjalan tanpa henti.

Dalam bahasa Go, implementasi rekursif cukup sederhana dan mudah dibaca. Dengan latihan seperti pada soal guided dan unguided, pemahaman terhadap rekursif akan semakin kuat dan bisa diterapkan dalam berbagai kasus pemrograman yang lebih kompleks.

